


LISTEN to Your Preferences

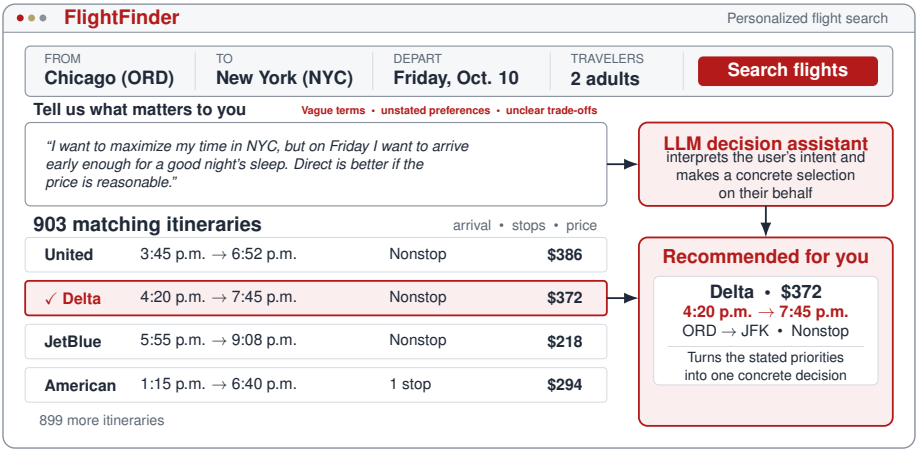
An LLM Framework for Multi-Objective Selection

Adam S. Jovine*[1], Tinghan Ye*[2], Francis Bahk[1], Jingjing Wang[1]
Matthew T. Ford[1], David B. Shmoys[1], Peter I. Frazier[1]

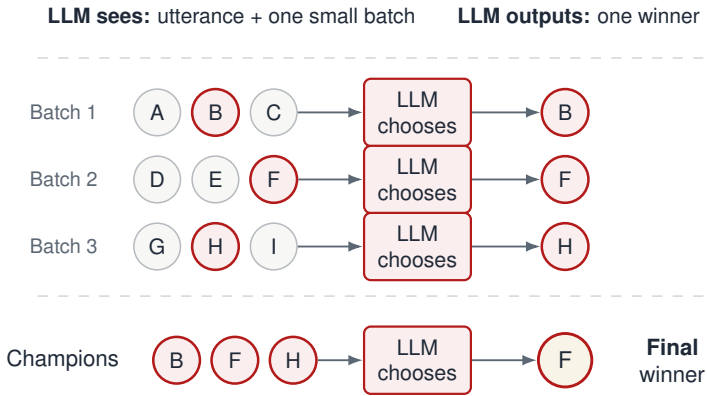
[1]Cornell University [2]Georgia Institute of Technology *Equal contribution

IJCAI–ECAI 2026

Goal: turn imprecise preference utterances into decisions

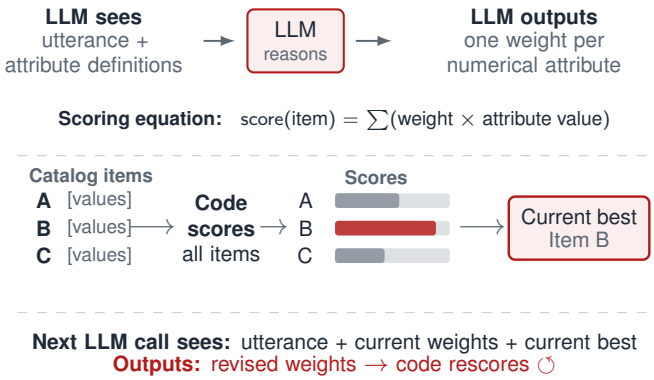


LISTEN-T: tournament of choices

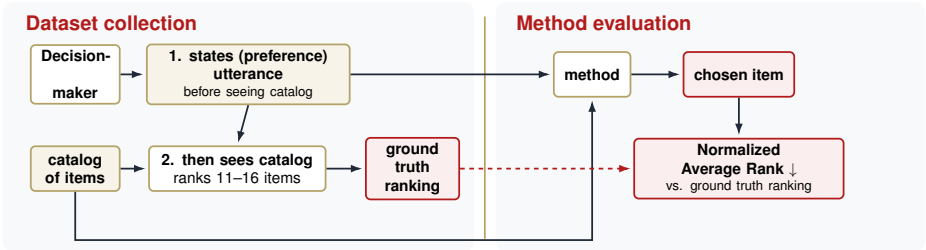


Code saves the three batch champions, then forms the final playoff.

LISTEN-U: refines a utility function



Data Collection and Method Evaluation



4 preference scenarios
2 flights • headphones

exam scheduling

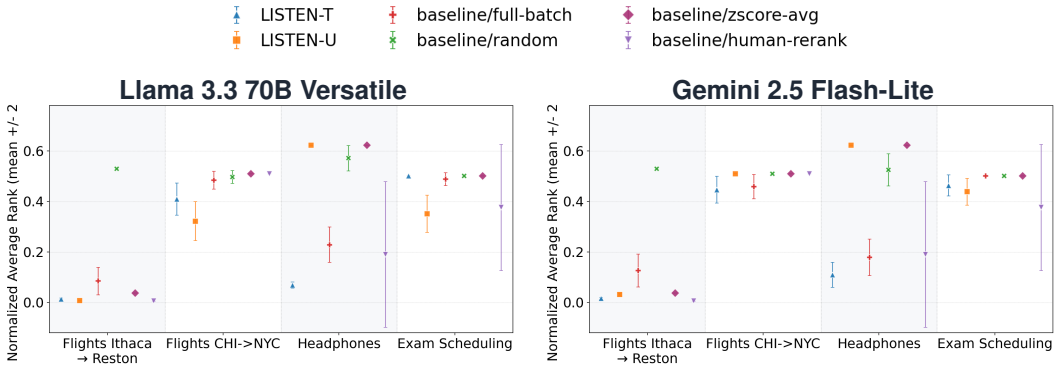
25 LLM calls
per replication
LISTEN-U: 1 initial + 24 refinements
LISTEN-T: 24 batches (32 items) + playoff

40 replications

4 baseline methods
Full Batch • random
equal-weight z-score

4 human rerankers

LISTEN typically outperforms Full Batch and other humans



LISTEN-U improves on headphones when preferences are more linear

Concordance = share of random linear utilities that select a ground-truth-ranked item. **Higher means more linear.**

Original Headphones
Hard requirements, including a ≥30-hour battery.

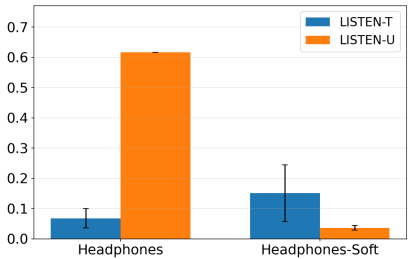
Concordance = **0.055**

→

Headphones-Soft
Same goals as preferences; no hard requirements.

Concordance = **0.244**

Normalized Average Rank (lower is better; mean ±2 SE)



Summary

- The iterative **LISTEN** methods typically outperform both **human rerankers**, who select from the catalog using the utterance, and **Full Batch**, which gives the entire catalog to the LLM at once.
- **LISTEN-U** works better when a linear utility model is a reasonable fit and may be more computationally efficient for large catalogs.
- **LISTEN-T** is a robust performer when preferences do not fit that linear structure.

Backup: exact LISTEN-U initialization template

Braced fields are filled at runtime; shown verbatim from the released `header_then_task_v1` template.

Context, task, and response schema

```
{scenario_header}

Your task: Define a utility function by assigning
      weights to these metrics, reflecting
      {utility_priorities}. {utility_task_body}

Return your response in this EXACT JSON format (include
      ALL listed metric keys):
{metric_weights_json}
```

Output contract

```
STRICT OUTPUT RULES:
- Output ONLY a valid JSON object. No code fences. No
      prose. No Python.
{utility_weight_rule}
- Do not include any additional keys beyond weights,
      formula, description.
{utility_key_rule}
- End your output with the delimiter: <END_JSON>
```

Runtime expansion supplies the scenario header and metric keys; the full policy guidance enters refinement calls.

Backup: exact LISTEN-U refinement template

The current weights and their best-scoring item become the next call's state.

Current state and critique

```
You are refining a utility function for
      {utility_domain} (iteration {iteration}).

CURRENT UTILITY FUNCTION (that produced the
      {best_item_ref} below):
Weights: {weights}
Formula: {formula}
Description: {description}

BEST {best_item_found} with the above utility function:
{best_solution}

ANALYSIS:
{analysis_questions}

Your task: Based on this {best_item_ref}, adjust the
      utility function weights if needed.
Consider:
{policy_guidance}
```

Required action and output

```
Return your response in this EXACT JSON format (include
      ALL listed metric keys):
{metric_weights_json_escaped}

STRICT OUTPUT RULES:
- Output ONLY a valid JSON object. No code fences. No
      prose. No Python.
{utility_weight_rule}
- Do not include any additional keys beyond weights,
      formula, description.
{utility_key_rule_refinement}
- If no change is needed, return the previous weights
      unchanged in the JSON.
- End your output with the delimiter: <END_JSON>
```

Backup: exact LISTEN-T comparison template

The same template is used for preliminary batches and the final playoff; only the options change.

```
{scenario_header}

Your task: Identify the SINGLE BEST {comparison_noun} from the options below.
=== ITEMS TO COMPARE (JSON) ===
Return EXACTLY these numbers when reasoning; do not infer or re-calc.
```json
{minified JSON containing options labeled A, B, ... and their metrics}
```

Do not explain. Using ONLY the JSON metrics above, respond with ONLY the single letter of your
     choice from: A, B, ...
```

Action: return one option label. The paper writes this contract as `FINAL A`; the released prompt builder requests the bare label `A`, and its parser accepts both forms.